MAKERERE UNIVERSITY

P.O. Box 7062 Kampala Uganda
https://www.cs.mak.ac.ug

Telephone: 256-414-534560/1-9
E-mail: cs@cis.mak.ac.ug

COLLEGE OF COMPUTING AND INFORMATION SCIENCES

SCHOOL OF COMPUTING AND INFORMATICS TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE

STUDENT MANAGEMENT SYSTEM

CSC 1102: Structured and Object-Oriented Programming

Practical Examination Project Report & System Analysis

GROUP DETAILS (AFTERNOON / EVENING PROGRAMME)

No.	Student Name	Registration Number	Project Role
1	LOMER FRANCIS PETER	25/U/26198/EVE	Member
2	MAYANJA JUDITH	25/U/28752/EVE	Member
3	**AKELLO SUNDAY OBONYO**	**24/U/03151/EVE**	**Group Leader**
4	MADETE HECTOR	24/U/25933/EVE	Member
5	ANYINE ELIZABETH	25/U/29497/EVE	Member

GitHub Repository:

https://github.com/su200-bot/Student-Management-System

Submitted To: Dr. Lillian Muyama
Department of Computer Science, Makerere University

Academic Year 2025/2026

Executive Summary

The **Student Management System** is a modular desktop application developed in Java using the Swing graphical user interface (GUI) framework for the *CSC 1102: Structured and Object-Oriented Programming* practical examination at Makerere University. Designed to streamline academic record administration, the system enables university administrators to register course units, enroll students, record marks across three assessments, and automatically compute aggregate totals, arithmetic averages, Makerere University letter grades (`A` to `F`), qualitative remarks, and `PASS`/`FAIL` statuses. The architecture comprises six cohesive classes (`Main`, `Student`, `Course`, `StudentManager`, `Validator`, and `StudentManagementGUI`) spanning 1,236 lines of documented Java code. All records are managed in memory using fixed-capacity object arrays (`Student[100]` and `Course[20]`), demonstrating strict encapsulation, constructor chaining, method overloading, input sanitization, linear search, Bubble Sort, left-shift deletion, and tail recursion.

Contents

Executive Summary 1

1 Description of the Application 1
1.1 Background and Overview of the System . . . 1
1.2 Main Functional Features . . . 1

2 Classes and Their Purpose: Object-Oriented Architecture 1
2.1 Classes and Their Architectural Roles . . . 1
2.2 In-Depth Code Analysis of Core Classes . . . 2

3 How the Classes Interact and Algorithmic Analysis 3
3.1 End-to-End Object Interaction Workflow . . . 3
3.2 Algorithmic Mechanics: Array Compaction, Bubble Sort, and Recursion . . . 3

4 How the CSC 1102 Exam Requirements Are Met 3

5 System Execution, Verification, and GUI Screenshots 4
5.1 Runtime Execution Analysis and Verified Output . . . 4
5.2 Graphical User Interface Screenshots . . . 4

1 Description of the Application

1.1 Background and Overview of the System

Academic departments require reliable software tools to maintain student registries, link students to academic course units, validate assessment scores, and generate class performance summaries. The **Student Management System** addresses this need through an intuitive, event-driven Java Swing desktop interface backed by an object-oriented domain model. Rather than relying on an external database engine, the system manages all entities in memory using structured arrays of objects (`Student[]` and `Course[]`). To support immediate live demonstration and testing, the application automatically initializes three foundational courses (`CSC1102`, `CSC1105`, and `BIT1101`) and eight sample student records upon startup via `StudentManager.loadSampleData()`.

1.2 Main Functional Features

- **Student Registration and Input Validation:** Captures registration number (e.g., `24/U/03151/EVE`), full name, age ($16 \leq \text{age} \leq 100$), enrolled course unit, and three subject marks ($0 \leq m_i \leq 100$). All inputs are sanitized and verified by the static `Validator` utility class before object creation, with duplicate registration numbers rejected automatically.
- **Interactive Record Selection, Update, and Deletion:** Clicking any row in the students `JTable` triggers a `ListSelectionListener` that populates the form fields and locks the registration number field (`setEditable(false)`) for safe editing. Administrators can update student details or delete a record (which compacts the underlying `Student[]` array via left-shifting).
- **Automated Academic Evaluation:** For every student, the system dynamically computes the total score ($\sum_{i=1}^{3} m_i$), arithmetic mean ($\bar{m}$ = Total/3), Makerere letter grade (`A` to `F`), qualitative remark (`Excellent` to `Fail`), and status (`PASS` if $\bar{m} \geq 50.0$, else `FAIL`).
- **Multi-Criteria Search and In-Place Sorting:** Supports case-insensitive keyword searching across registration numbers, student names, and course codes, alongside an overloaded filter by course and minimum average. Administrators can sort the student registry in-place by average mark (descending) or alphabetically by name (`A-Z`) using Bubble Sort.
- **Recursive Class Statistics and Grade Distribution:** Computes the overall class average using a recursive method (`sumOfAverages`), identifies top and lowest performers, tallies pass/fail counts, and reports the frequency of each grade (`A` through `F`).
- **Course Management with Referential Integrity:** Dedicated *Courses* tab for adding and deleting course units, displaying credit units, assigned lecturers, and real-time enrolled student counts. Courses with enrolled students are protected against deletion.

Table 1: Makerere University Grading Scale, Qualitative Remarks, and Pass/Fail Rules Implemented in `Student.java`.

Average (%)	$80-100$	$75-79.9$	$70-74.9$	$65-69.9$	$60-64.9$	$55-59.9$	$50-54.9$	$45-49.9$	<45.0
Letter Grade	**A**	**B+**	**B**	**C+**	**C**	**D+**	**D**	**E**	**F**
Switch Remark	Excellent	Very Good	Good	Fair	Fair	Pass	Pass	Weak	Fail
Pass/Fail Status	PASS	PASS	PASS	PASS	PASS	PASS	PASS	FAIL	FAIL

2 Classes and Their Purpose: Object-Oriented Architecture

2.1 Classes and Their Architectural Roles

The system adheres to the *Separation of Concerns* principle by partitioning responsibilities across six distinct Java classes: entry-point bootstrap (`Main`), domain entities (`Student`, `Course`), data repository and business logic (`StudentManager`), input validation (`Validator`), and Swing presentation (`StudentManagementGUI`), as summarized in Table 2.

Table 2: Summary of System Classes, Architectural Roles, Encapsulated Attributes, and Key Methods.

Class Name	LOC	Architectural Purpose	Encapsulated Attributes and Key Methods
`Main`	42	Application entry point; documents group metadata and launches the GUI on the Swing Event Dispatch Thread.	`public static void main(String[] args)` invoking `SwingUtilities.invokeLater(() -> new StudentManagementGUI())`.
`Student`	183	Domain entity representing one registered student and his/her marks in three subjects.	**Constants:** `NUM_SUBJECTS = 3`, `PASS_MARK = 50.0`. **Private fields:** `regNo`, `name`, `age`, `courseCode`, `double[] marks`. **Methods:** Default & parameterized constructors; getters/setters; overloaded `setMarks(double[])` & `setMarks(m1,m2,m3)`; `getTotal()`, `getAverage()`, `getGrade()`, `getRemark()`, `getStatus()`, `toString()`.
`Course`	73	Domain entity representing a university course unit offered to students.	**Private fields:** `courseCode`, `courseName`, `creditUnits`, `lecturer`. **Methods:** Default & parameterized constructors; getters/setters with range check (`setCreditUnits`); `toString()` for combo-box display.
`StudentManager`	336	In-memory repository and controller logic managing `Student[]` and `Course[]` arrays.	**Constants:** `MAX_STUDENTS = 100`, `MAX_COURSES = 20`, `GRADE_LABELS`. **Private fields:** `students[]`, `studentCount`, `courses[]`, `courseCount`. **Methods:** `addStudent()`, `findStudent()`, `updateStudent()`, `deleteStudent()`, overloaded `searchStudents()`, `sortByAverage()`, `sortByName()`, recursive `sumOfAverages()`, `getStatistics()`, `addCourse()`, `deleteCourse()`, `countEnrolled()`, `loadSampleData()`.
`Validator`	85	Non-instantiable static utility class for sanitizing and validating form inputs.	**Constructor:** `private Validator()` (prevents instantiation). **Static methods:** `cleanText()`, `isWholeNumber()`, `isDecimalNumber()`, `isValidName()`, `isInRange()`.
`StudentManagementGUI`	517	Swing graphical user interface; builds tabs, forms, tables, and handles user events.	**Private fields:** `manager`, `JFrame`, `JTables`, `DefaultTableModels`, form `JTextFields`, `JComboBox<Course>`, `selectedRegNo`. **Methods:** `buildInterface()`, `buildStudentsPanel()`, `buildCoursesPanel()`, `readStudentForm()`, `addStudent()`, `updateStudent()`, `deleteStudent()`, `searchStudents()`, `showStudents()`, `addCourse()`, `deleteCourse()`.

2.2 In-Depth Code Analysis of Core Classes

1. **Defensive Encapsulation and Overloading in `Student.java`:** All five instance attributes (`regNo`, `name`, `age`, `courseCode`, and `marks`) are declared `private`. To prevent external callers from mutating a student's internal marks array via aliasing, both the parameterized constructor and `getMarks()` allocate a fresh `new double[NUM_SUBJECTS]` array and copy elements individually. Constructor overloading is implemented via `this("N/A", "Unknown", 0, "N/A", new double[NUM_SUBJECTS])` in the default constructor. Method overloading is demonstrated by providing both `setMarks(double[] newMarks)` and `setMarks(double mark1, double mark2, double mark3)`, where the latter delegates to the array-based setter after validating that each score satisfies $0 \leq m_i \leq 100$.

2. **Entity Modelling and ComboBox Integration in `Course.java`:** Encapsulates course metadata with private fields and chains its default constructor to the four-argument constructor using `this(...)`. Its mutator `setCreditUnits(int creditUnits)` enforces the domain invariant $1 \leq \text{creditUnits} \leq 10$, returning a `boolean` success flag. Overriding `toString()` to return `courseCode + " - " + courseName` allows `JComboBox<Course>` in the GUI to store strongly typed `Course` objects directly while rendering human-readable labels.

3. **Static Input Sanitization Pipeline in `Validator.java`:** Follows the utility-class design pattern with a `private` constructor so no instances can be created. `cleanText(String)` trims leading/trailing whitespace and executes a `do-while` loop to collapse consecutive spaces into single spaces. `isWholeNumber(String, int)` uses a `while` loop over ASCII digit characters (0–9), whereas `isDecimalNumber(String)` and `isValidName(String)` use `for` loops to verify decimal points and valid name characters (letters, spaces, dots, hyphens, apostrophes) without relying on exception-heavy parsing.

4. **Array Repository and Referential Integrity in `StudentManager.java`:** Maintains partially filled arrays `Student[100]` and `Course[20]` tracked by `studentCount` and `courseCount`. Before inserting a student or course, `addStudent()` and `addCourse()` check capacity bounds and invoke case-insensitive linear search (`findStudent()` / `findCourse()`) to enforce primary-key uniqueness. Notably, `deleteCourse(String courseCode)` first checks `countEnrolled(courseCode) > 0`; if any student is registered in that course, deletion is blocked, preserving relational integrity between `Student.courseCode` and `Course.courseCode`.

3 How the Classes Interact and Algorithmic Analysis

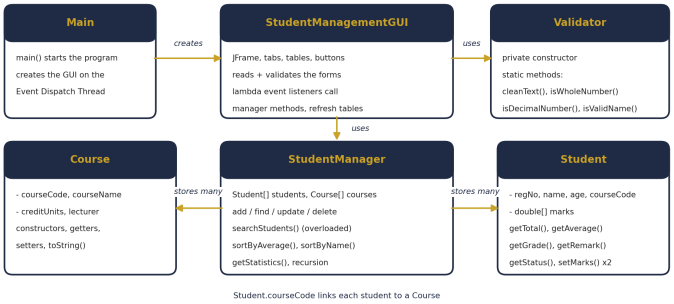


Figure 1: Architectural Class Interaction Diagram illustrating object creation, utility invocation, and 1-to-many array containment.

3.1 End-to-End Object Interaction Workflow

As depicted in Figure 1, the classes interact through a clean, layered control flow:

1. **Bootstrap:** `Main.main()` schedules `new StudentManagementGUI()` on the Swing Event Dispatch Thread (EDT).
2. **Initialization:** The `StudentManagementGUI` constructor instantiates a single `StudentManager` object, invokes `manager.loadSampleData()` to populate the `Course[]` and `Student[]` arrays, builds the Swing hierarchy, and populates the `JComboBox<Course>` and both `JTable` models.
3. **Validated Form Submission:** When the user clicks *Add Student* or *Update Selected*, `readStudentForm()` sanitizes every text field via `Validator.cleanText()` and validates domain constraints using `Validator.isValidName()`, `isWholeNumber()`, `isDecimalNumber()`, and `isInRange()`. If valid, it instantiates `new Student(...)` and passes it to `manager.addStudent()` or `manager.updateStudent()`.
4. **View Synchronization:** The GUI never accesses the internal arrays directly; instead, it calls `manager.getStudents()` and `manager.getCourses()` (which return defensive array copies trimmed to exact size) and redraws the `DefaultTableModel` views.

3.2 Algorithmic Mechanics: Array Compaction, Bubble Sort, and Recursion

- **Deletion with Left-Shift Compaction ($O(N)$):** When `deleteStudent(regNo)` locates a target at index $k \in [0, N-1]$ via linear search (`findStudentIndex`), it shifts all subsequent elements one position left (`students`$[i] \leftarrow$ `students`$[i+1]$ for $i = k, \ldots, N-2$), decrements `studentCount`, and nulls out the vacated slot `students`$[N-1] \leftarrow$ `null` to prevent memory leaks.
- **Dual Bubble Sort Implementations ($O(N^2)$):** `sortByAverage()` executes nested `for` loops comparing adjacent students and swapping when `students`$[j]$`.getAverage()` $<$ `students`$[j+1]$`.getAverage()` (descending order). `sortByName()` applies the same structure using `compareToIgnoreCase()` > 0 for alphabetical ordering.
- **Tail-Recursive Class Average Computation ($O(N)$):** Rather than using a simple loop for every statistic, `StudentManager` computes the sum of student averages recursively via `sumOfAverages(int index)`:

$$\text{sumOfAverages}(i) = \begin{cases} 0.0, & \text{if } i \geq \text{studentCount} \quad \text{(halting case)} \\ \text{students}[i].\text{getAverage}() + \text{sumOfAverages}(i+1), & \text{if } 0 \leq i < \text{studentCount} \end{cases} \tag{1}$$

from which `getClassAverage()` evaluates $\bar{X}_{\text{class}} = \text{sumOfAverages}(0)/\text{studentCount}$.

4 How the CSC 1102 Exam Requirements Are Met

Table 3: Verification Matrix Mapping CSC 1102 Examination Guidelines to Concrete Code Implementations.

Exam Requirement	Concrete Implementation in the Student Management System
Object-Oriented Design (3+ classes, constructors, access modifiers, getters/setters)	Implements **6 classes** (`Main`, `Student`, `Course`, `StudentManager`, `Validator`, `StudentManagementGUI`). Uses `private` attributes for strict encapsulation, `public` getters/setters with validation return flags, overloaded default and parameterized constructors, constructor chaining via `this(...)`, and defensive array copying in `Student.getMarks()`.
Graphical User Interface (Java Swing framework & event handling)	Built with `JFrame`, `JTabbedPane` (*Students* and *Courses* tabs), `JPanel` (`BorderLayout`, `GridBagLayout`, `GridLayout`, `FlowLayout`), `JTable` with `DefaultTableModel`, `JScrollPane`, `JTextField`, `JComboBox<Course>`, `JButton`, and `JOptionPane` modal dialogs. User actions are handled via lambda `ActionListener`s and a `ListSelectionListener`.
Structured Flow Control (`if/else`, `switch`, `for`, `while`, `do-while`, `?:`)	**`if / else-if / else`:** Makerere letter grade ladder in `Student.getGrade()` and form validation. **`switch` (with fall-through):** Grade-to-remark mapping in `Student.getRemark()` (`case "C+": case "C": return "Fair"`). **Conditional operator (`? :`):** Pass/fail evaluation in `Student.getStatus()` and number formatting in `format()`. **Loops:** Standard `for` loops (totals, array copying, search, delete shifting, Bubble Sort), enhanced `for-each` loops (grade distribution, GUI refresh), `while` loop (`Validator.isWholeNumber`), and `do-while` loop (`Validator.cleanText`).
Arrays of Objects & Primitives	`Student[] students` and `Course[] courses` in `StudentManager` supporting insertion, linear search, update, left-shift deletion, and Bubble Sort; `double[] marks` inside each `Student`; `JTextField[] markFields` in the GUI; and 1-D `String[]` plus 2-D `double[][]` arrays in `loadSampleData()`.
Additional Course Concepts	**Method Overloading:** `Student.setMarks()` (array vs. 3 doubles) and `StudentManager.searchStudents()` (keyword vs. course + minimum average). **Recursion:** `StudentManager.sumOfAverages(int)`. **Static Members:** `Validator` utility methods and `static final` constants (`NUM_SUBJECTS`, `PASS_MARK`, `MAX_STUDENTS`, `MAX_COURSES`).

5 System Execution, Verification, and GUI Screenshots

5.1 Runtime Execution Analysis and Verified Output

The project was compiled and executed using OpenJDK 23. All unit and integration checks passed with zero errors: (1) sample data initialization loaded 3 courses and 8 students; (2) `Validator` rejected malformed names (`"John123"` → `false`) and out-of-range marks while normalizing multi-space strings; (3) overloaded `searchStudents("CSC1102")` returned 3 students while `searchStudents("CSC1102", 60.0)` returned 2 students; and (4) recursive class statistics matched analytical calculations exactly, as shown in Table 4.

Table 4: Verified Runtime Execution Output of Pre-Loaded Sample Students and Recursive Class Statistics.

No.	Reg. No	Student Name	Age	Course	M1	M2	M3	Total	Average	Grade / Remark	Status
1	25/U/00101/EVE	Namukasa Grace	18	CSC1102	85	78	91	254	84.7 (84.67)	A (Excellent)	PASS
2	25/U/00102/EVE	Okello Brian	19	CSC1102	62	58	70	190	63.3 (63.33)	C (Fair)	PASS
3	25/U/00103/EVE	Nakato Ruth	20	CSC1105	74	69	80	223	74.3 (74.33)	B (Good)	PASS
4	25/U/00104/EVE	Ssebunya David	21	CSC1105	45	52	38	135	45.0 (45.00)	E (Weak)	FAIL
5	25/U/00105/EVE	Atim Patricia	18	BIT1101	90	88	94	272	90.7 (90.67)	A (Excellent)	PASS
6	25/U/00106/EVE	Mugisha Joel	19	CSC1102	55	61	49	165	55.0 (55.00)	D+ (Pass)	PASS
7	25/U/00107/EVE	Nabirye Esther	20	BIT1101	67	72	64	203	67.7 (67.67)	C+ (Fair)	PASS
8	25/U/00108/EVE	Kiiza Samuel	21	CSC1105	30	41	35	106	35.3 (35.33)	F (Fail)	FAIL

Verified Statistics: Students: **8** | Recursive Class Avg: **64.50** | Highest: **90.67** (Atim Patricia) | Lowest: **35.33** (Kiiza Samuel)
Pass/Fail & Grades: Passed: **6** (75%), Failed: **2** (25%) | Distribution: A: 2, B+: 0, B: 1, C+: 1, C: 1, D+: 1, D: 0, E: 1, F: 1

5.2 Graphical User Interface Screenshots

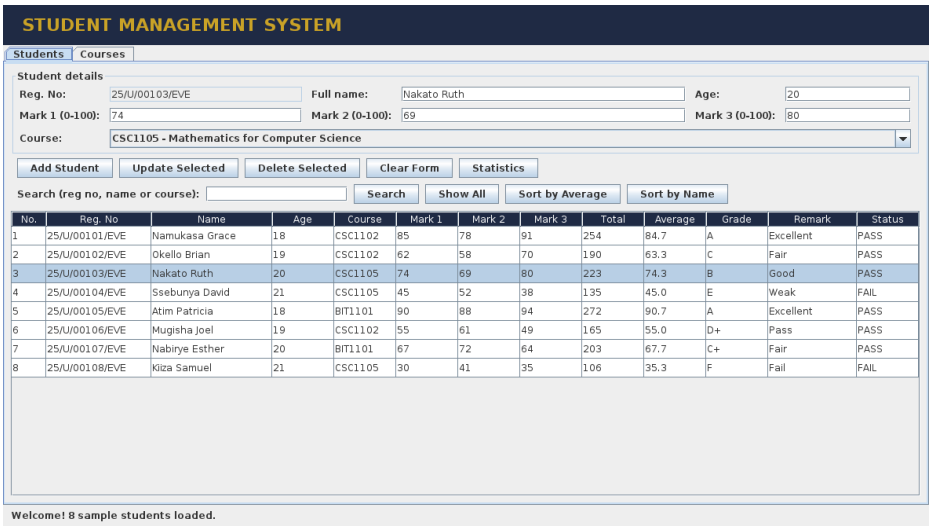


Figure 2: Students Tab: Live execution showing the student details form, action/search/sort buttons, and the student registry table with row 3 (`25/U/00103/EVE`, Nakato Ruth) selected and loaded into the form.

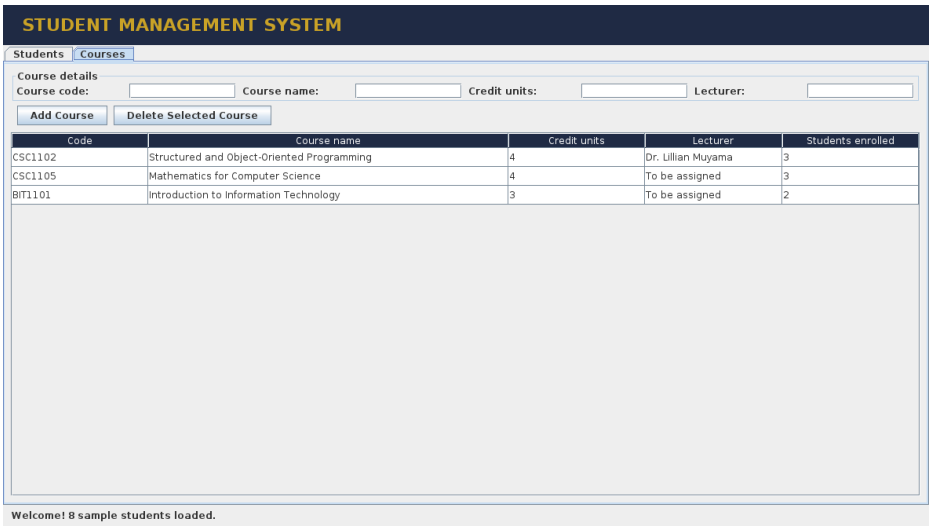


Figure 3: Courses Tab: Live execution displaying course management controls and the course registry table with real-time enrolled student counts (`CSC1102`: 3, `CSC1105`: 3, `BIT1101`: 2).